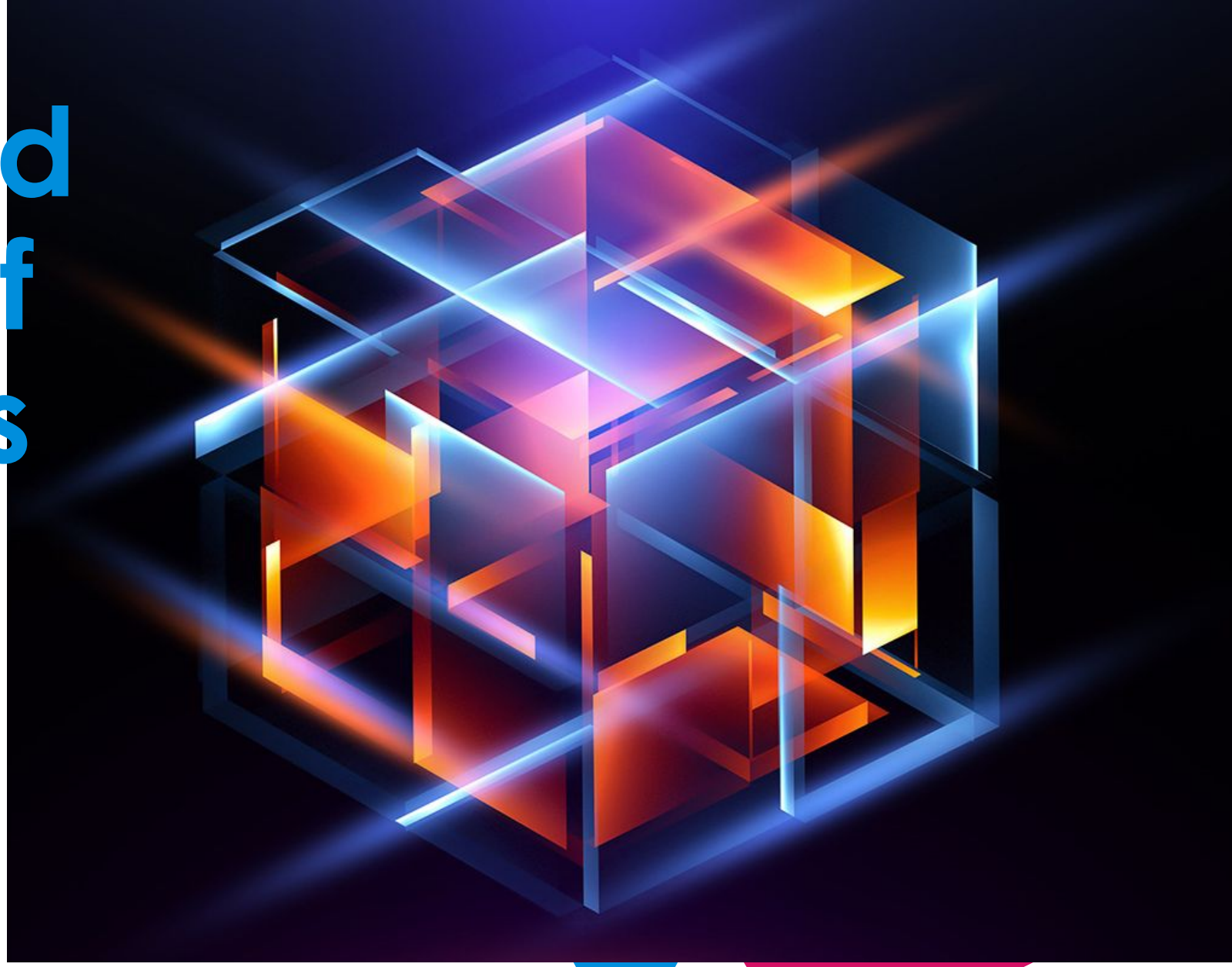
Desing and Analysis of Algorithms

Analyzing Recursive Algorithms



Agenda

Recursion

Recursion Tree Method

- **Steps:**

1. **Draw the Root:** Write down the cost of the non-recursive part of the function f(n) as the root of the tree.
2. **Branch Out:** Draw children representing the recursive calls (e.g., for 2T(n/2), draw two children, each with cost f(n/2)).
3. **Calculate Level Cost:** Sum the total cost of all nodes at each level of the tree.
4. **Determine Height:** Find the depth of the tree by determining how many levels it takes to reach the base case (usually T(1)).
5. **Sum Total Cost:** Add the costs of all levels together to get the final asymptotic complexity.

Example

- $T(n) = 2T(n/2) + n$
- **Root:** n
- **Level 1:** Two nodes of n/2 (Sum = n)
- **Level 2:** Four nodes of n/4 (Sum = n)
- **Height:** log n levels.
- **Total:** $n \log n = \Theta(n \log n)$

Substitution Method

- **Steps:**

1. **Expand:** Write out the recurrence, then substitute the function into itself to "unroll" it a few times.
2. **Identify Pattern:** Look for the mathematical pattern emerging after k expansions. Express the equation purely in terms of n and k.
3. **Find Base Case k:** Set the inner recursive term equal to your base case (e.g., n - k = 1) to solve for k.
4. **Substitute and Solve:** Plug the value of k back into your pattern equation and solve the resulting mathematical series (e.g., arithmetic or geometric progression).

Example

$T(n) = T(n-1) + n$

- **Step 1:** $T(n) = [T(n-2) + (n-1)] + n$
- **Step 2 (Pattern):** $T(n) = T(n-k) + n + (n-1) + \ldots\ldots + (n-k+1)$
- **Step 3:** Base case $T(1)$ occurs when $n - k = 1$, $k = n - 1$.
- **Step 4:** $T(n) = 1 + 2 + 3 + \ldots\ldots + n = n(n+1)/2 = \Theta(n^2)$

Master Theorem

Steps:

1. **Check Format:** Ensure the recurrence fits the standard form: $T(n) = aT(n/b) + f(n$ where $a \geq 1$ and $b > 1$.
2. **Calculate Watershed:** Compute $n^{\log_b a}$.
3. **Compare:** Compare $f(n)$ to $n^{\log_b a}$ asymptotically:
   - **Case 1:** If $f(n)$ is polynomially smaller, $T(n) = \Theta(n^{\log_b a})$.
   - **Case 2:** If they are equal (multiplied by $\log^k n$), $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
   - **Case 3:** If $f(n)$ is polynomially larger, $T(n) = \Theta(f(n))$.

Example

- $T(n) = 3T(n/2) + n^2$
- **Identify:** $a = 3, b = 2, f(n) = n^2$
- **Watershed:** $n^{\log_2 3} \approx n^{1.58}$
- **Compare:** $f(n) = n^2$ is polynomially strictly larger than $n^{1.58}$ (Case 3).
- **Result:** $T(n) = \Theta(n^2)$

Changing Variables

- **Steps:**

1. **Identify Substitution:** Choose a new variable to simplify the recursive arguments (e.g., if you see sqrt(n), let $n = 2^m$).
2. **Rewrite Equation:** Replace all instances of n with the new variable.
3. **Rename Function:** Define a new function (e.g., S(m)) to replace the transformed T function, converting it into a standard linear recurrence.
4. **Solve:** Use standard methods (like the Master Theorem) on the new function S(m).
5. **Revert Variable:** Substitute the original variable back into the final answer.

Example

T(n) = 2T($\sqrt{n}$) + log n

- **Step 1 & 2:** Let $n = 2^m$ (so $m = \log n$). Equation becomes $T(2^m) = 2T(2^{m/2}) + m$.
- **Step 3:** Let $S(m) = T(2^m)$. Now, $S(m) = 2S(m/2) + m$.
- **Step 4:** Apply Master Theorem. $a = 2, b = 2, f(m) = m$. Watershed is $m^{\log_2 2} = m^1$. Since they match (Case 2), $S(m) = \Theta(m \log m)$.
- **Step 5:** Substitute m back out: $T(n) = \Theta(\log n \log \log n)$.

Questions

